

shape\_blocks

NAME

shape\_blocks

Shapes and places the shaping objects (physical blocks, power domains or voltage areas, and move bounds) with respect to constraints and utilization requirements. If no utilization constraint is set on the shaping object, the design's utilization will be used. The command uses macro packing to place the top-level macros. If the block grid is created for the block before shaping, it will also align the block shape to the specified block grid. If the placement abstracts of blocks have not been created before this command, shape\_blocks will automatically create them.

SYNTAX

```
int shape_blocks
    [-channels true | false]
    [-constraint_file file_name]
    [-incremental congestion_driven | target_utilization_driven]
    [-pg_strategy pg_strategy_name]
    [-host_options host_options_name]
```

Data Types

|                   |        |
|-------------------|--------|
| file_name         | string |
| pg_strategy_name  | string |
| host_options_name | string |

ARGUMENTS

-channels true | false

Specifies whether channels are created between blocks. Channels can be inserted between blocks or between blocks and the core boundary. By default, the option is true and the command inserts channels.

If channels are required, the channel size is automatically calculated based on the required routing resources among hierarchies and the allowed utilization of each physical hierarchy.

You can also specify the required minimal channel size with the set\_shaping\_options -min\_channel\_size command. The larger value of user-specified channel size and automatically calculated channel size is used to create channels. When -min\_channel\_size is specified, the automatically calculated channel size is based only on the required routing resources, not on the allowed utilization.

-constraint\_file file\_name

Specifies the file that contains user constraints to control the channel sizes, aspect ratio, utilization, fixed shapes, and relative placement. The supported constraint formats are as follows.

o Constraint types

The following keywords define shaping constraints:

1. block related keywords
  - o allowed\_orientation
  - o aspect\_ratio
  - o utilization
  - o boundary
2. edge related keywords
  - o boundary\_channel\_size
  - o channel\_size
  - o guard\_band
3. grouping related keywords
  - o define\_group
4. relative placement related keywords
  - o arrange\_in\_box
  - o arrange\_in\_array

Constraints can take parameters, which is illustrated in the following example of defining channels around a block CPU instantiated within block B:

```
block B {
    channel_size {
```

```
// This constraint only applies to sub-block CPU.
block_inst CPU;
```

```
// Default: any channel surrounding CPU should be 20u wide.
// Except left and right size of CPU // which are between 30u
and 50u. min = 20, max = 20; left, right: min = 30, max = 50;
} // Other constraints within the scope of block B go here }
```

#### o Scoping and grouping rules

Global constraints applied to any block are defined at the global scope. These constraints do not have an encompassing block scope. The only constraints that can be defined in global scope are: `boundary_channel_size`, `channel_size`, `guard_band` and `utilization`. For example

```
utilization { target: 0.3; }
```

specifies a target utilization of 30% for any shapeable region in this design, unless it is overridden by any later constraint.

`guard_band` and `channel_size` can also be specified in the global scope. For example

```
guard_band { width: 2; height: 3; }
channel_size { size = 5; }
block TOP {
    // define constraints in block scope
    .....
}
```

specifies a `guard_band` for any voltage area of 2u in width and 3u in height. It also specifies the channel size of 5u between any blocks/VAs.

The default guard band size is zero. For non-abutted floorplan (i.e., option `-channels true` is applied), the default channel size is calculated to minimize the channel routing congestion.

The space between two neighboring blocks is sum of the outer keepouts and channel size for the blocks. The space between two neighboring voltage areas is sum of guard band sizes and channel size.

The constraints defined in the global scope can be overwritten by the constraints defined in the block scope, as shown in the following example.

Constraints applicable to objects within a particular block are defined inside a block scope:

```
block <reference_name> {
    constraint { }
    constraint { }
    .....
}
```

The reference name identifies the block being constrained. The constraints defined within the block scope can only refer to objects visible in the physical block, or to the physical block itself using the keyword `current_block`. The constraints cannot refer to objects of another physical block.

Constraints specific to a voltage area in a given block are specified using a voltage area scope. For example

```
guard_band { width: 1; height: 1; }
block BLK1 {
    voltage_area VAS1 {
        contents: voltage_area VAS2, block_inst BLK2;

        // VAS1 and nested voltage areas, like VAS2 get 40u guard band
        // even though the global guard band is 1u. This constraint is
        // limited to voltage areas of this block, BLK1.
        guard_band { width: 40; height: 40; }

        ...
    }
    guard_band {
        voltage_area VA1;
        width: 20; height: 20;
    }
}
```

```
}
```

Specifies the following:

1. The voltage area VAS2 and block BLK2 are to be shaped inside VAS1
2. A guard band of 40u for VAS1 and any voltage area nested inside (but within the current block)
3. A guard band of 20u for voltage area VA1
4. A guard band of 1u for other voltage areas

You can define new groups of objects like block instances, voltage areas and other groups with the `define_group` constraint.

```
block BLK1 {
  define_group GRP1 {
    // E, F, MB1, and VA3 make up group GRP1
    contents: block_inst E, block_inst F,
             move_bound MB1,
             voltage_area VA3;

  }

  arrange_in_array {
    // Refers to the group defined above
    contents: group GRP1,
             block_inst J;
    direction: east;
  }
}
```

Groups cannot have partial overlap, but nesting is allowed. For example, the following constraint specifies that two blocks, E and F, must be arranged from top to bottom, and shaped along VA3:

```
block BLK1 {
  define_group GRP1 {
    contents: block_inst E, block_inst F,
             voltage_area VA3;

    define_group GRP2 {
      contents: block_inst E, block_inst F;
      arrange_in_array {
        contents: block_inst E, block_inst F;
        direction: south;      // from top to bottom
      }
    }
  }
  // GRP1 shadows E, F and VA3. Constraints at this point
  // can refer to GRP1, (not to E, F and VA3) Those have
  // to be defined inside GRP1 or GRP2.
}
```

In the scope where a group is defined, the group will shadow the objects it contains to prevent inadvertent group overlap. In the example above, GRP1 will shadow E, F and VA3.

## o Constraints

### A. Boundary channel constraint

The `boundary_channel_size` constraint reserves a space between a block or voltage area and its nested regions. The `boundary_channel_size` and `channel_size` constraints applying to the same channel are combined by taking the maximum of the two. When a `boundary_channel_size` constraint is defined inside the global scope, it applies to the boundary inside any block or voltage area. Alternatively, the `boundary_channel_size` is defined within a block scope, and is applied to the current\_block or to a voltage area, specifying the boundary channel's size along the inside of that region's boundary. For example,

```
block CPU {
  boundary_channel_size {
    current_block;
    left: min = 3, max = 5;
    right, bottom: min = 5, max = 99;
  }
}
```

specifies that any voltage area or block instance shaped within CPU is to stay at least 3u to 5u from the left side of the boundary, and 5u to 99u from the right and bottom side of the boundary.

## B. Utilization constraint

Utilization constraints specify a minimum, target or maximum cell utilization of a shapeable region. For example, requesting a 53% target utilization and setting an upper limit of 70% and a lower limit of 40% is achieved by:

```
utilization {
    target: 0.53;
    max: 0.7;
    min: 0.4;
}
```

Utilization of block instances cannot be specified individually, because multiple instantiations (MIB's) must follow the same constraint. Instead, you define the constraint for the reference design, say X:

```
block X {
    utilization { target: 0.4; }
}
```

A utilization constraint that does not explicitly name any objects will be applied to the shapeable regions in the current scope (limited to the current block). In the next example a 60% utilization target is set for VA1, VA2 and VA3 (as both are part of G2), but not to block\_inst B.

```
define_group G2 { contents: voltage_area VA2, voltage_area VA3; }
define_group G1 {
    contents: voltage_area VA1, group G2, block_inst B;
    utilization { target: 0.6; }
}
```

The above is equivalent to:

```
define_group G2 { contents: voltage_area VA2, voltage_area VA3; }
define_group G1 { contents: voltage_area VA1, group G2, block_inst B; }
utilization { group G1; target: 0.6; }
```

## C. Aspect ratio constraint

The target aspect ratio of a region can be specified for blocks that are placed with an `arrange_in_box` constraint:

```
block RESET {
    aspect_ratio {
        current_block;
        target: width = 3, height = 4;
    }
}
```

## D. Guard band constraint

Guard band constraints only affect shapeable voltage areas, and control the width and height of their guard bands. If you specify a voltage area, the constraint will apply to that voltage area only (not recursively). If you specify a group, the constraint will apply to the voltage areas of that group (not recursively). In the example below, a guard band of 20 is assigned to VA3 and VA6.

```
block CPU1 {
    define_group GRP1 { contents: voltage_area VA6, block_inst L; }
    guard_band { voltage_area VA3, group GRP1; width: 20; height: 20; }
}
```

If you leave out a list of regions, the constraint will be applied recursively to any voltage area within the encompassing scope and current block. If the encompassing scope is a voltage area, it is included as well. The example below, shows how a guard band of 10u is assigned to all the voltage areas within GRP2 and below (within the same block): VA1, VA2, VA4 and VA5. It also shows how nesting between voltage areas and between move bounds.

```
block CPU1 {
    // define nesting between voltage areas
    voltage_area VA1 {
        contents: voltage_area VA4, voltage_area VA5;
    }
}
```

```

// define nesting between move bounds
move_bound mb1 {
    contents: move_bound mb2;
}
define GRP2 {
    contents: voltage_area VA1, voltage_area VA2, block_inst K;
    guard_band { width: 10; height: 10; }
}
}

```

#### E. Channel size constraint

A `channel_size` constraint specifies the size of a channel between regions.

```

channel_size {
    // Primary objects.
    block_inst CPU1, block_inst CPU2;

    // Secondary (neighboring) objects.
    neighbors: current_block;

    // Any channel between block (CPU1 or CPU2) and current_block is 10.
    // For the channel along the left, right side of CPU1 (or CPU2) and
    // neighboring the boundary of current_block,
    // limit the size of the channel to [3, 5].
    size=10;
    left, right: min=3, max=5;
}

```

A `channel_size` constraint contains an object specifier followed by one or more range specifiers. Object specifiers can be:

1. empty, the channel size constraint applies to any object in the current scope.
2. a single list, specifying primary objects:
  - o `block_inst` followed by an identifier
  - o `voltage_area` followed by an identifier
  - o `move_bound` followed by an identifier
  - o `group` followed by an identifier
  - o `current_block` identifying the boundary of the encompassing block
3. between a primary object and its neighbor: two lists specified similarly as above. Applies to all possible pairs formed by an object from the primary list and an object from the secondary list.

A range specifier is an edge specifier followed by a limit specifier. They can be:

1. empty, in which case the channel size constraint applies to any edge.
2. side specific: any of `left`, `right`, `bottom`, `top` to identify the sides of the primary object.

A limit specifier defines the size range of the constrained channel. It can be one of the following three:

1. `min = <IDENTIFIER>` : specifies a minimum size, maximum unconstrained.
2. `max = <IDENTIFIER>` : specifies a maximum size, minimum is bounded by default (0).
3. `min = <IDENTIFIER>`, `max = <IDENTIFIER>` : limits the minimum and maximum size of this channel.
4. `size = <IDENTIFIER>` : specifies an exact size.

The `channel_size` constraint specified last overwrites any previously defined `channel_size` constraints with equal object specifier. A more specific constraint takes precedence over less specific constraints. Channel size constraints defined inside a block scope are more specialized than channel size constraints defined at the global scope. Specialization is next determined by the object specifier, defined to have increasing order of precedence as follows: empty, object list, object list with neighbor list.

#### F. Relative placement constraint

The `arrange_in_array` and `arrange_in_box` constraints instruct the shaper to generate a specific region layout.

The `arrange_in_array` constraint has the `contents` list and a `direction`. The shaper places the regions in the `contents` list in an array following the specified order, growing the array in the specified direction. Valid directions are: `north`, `west`, `south`, `east`.

```

    block B {
        arrange_in_array {
            contents: block_inst A, voltage_area VA1, group G2;
            direction: west;
        }
    }
}

```

In above example, group A, VA1, and G2 are placed right-to-left in a horizontal array. Change the order to left-to-right by setting the direction to east.

The `arrange_in_box` constraint places the contents at the specific location. Optionally, the `arrange_in_box` constraint carries an alignment point (default at center). When an alignment point is specified, the shaper tries to align that point with the target location.

```

    voltage_area VA1 {
        arrange_in_box {
            block_inst A;
            // Optional alignment point, default is center.
            alignment_point: x = 1, y = 0;
            location: x = 0.5, y = 0.5;
        }
    }
}

```

In above example, block instance A is placed within VA1's shape. A's lower right corner (1, 0) is to be aligned with VA1's center.

In the following example, the shaper first shapes block C (if it does not already have a given shape) and then applies the `arrange_in_box` constraints in their order of specification. Finally, the `arrange_in_array` constraints are applied. In the example above, the shaper places A, GRP1 (= D, E, F, G, VA2). This leaves H and X to be shaped into the cavity defined by C minus the area assigned to A and GRP1. The shaper then arranges H on the left of X. The region layout within group GRP1 is also predetermined. A subgroup GRP2 consisting of block instances D and E is defined, and given a center location. The coordinates define a location within the encompassing region's bounding box, here GRP1, and range from (0, 0), the lower left corner, to (1,1), the upper right corner. Again, the `arrange_in_box` constraint takes precedence over the `arrange_in_array` constraint. Note that VA2 is left unconstrained. Finally, within group GRP2, the block instances D and E are arranged from top to bottom. This constraint is applied once GRP2 has been assigned a shape.

```

// Block C contains the following regions for shaping: [A, D, E,
// F, G, H, X, VA2]. Blocks D, E, F, G and VA2 should be kept together,
// other blocks can be placed without restriction
block C {

    define_group GRP1 {
        // The following regions make up group GRP1
        // They must be within C's scope.

        contents: block_inst D, block_inst E, block_inst F,
                block_inst G, voltage_area VA2;

        define_group GRP2 {
            arrange_in_array {
                contents: block_inst D, block_inst E;
                direction: south;
            }
        }

        // Order is important. First apply the arrange_in_box,
        // then apply the arrange_in_array constraint
        arrange_in_box {
            // Referring to D and E is not allowed here, because
            // they are grouped into GRP2
            group GRP2;
            location: x = 0.5, y = 0.5; // place in the middle
        }

        // Note: this implicitly defines an unnamed group of F and G.
        arrange_in_array {
            contents: block_inst F, block_inst G;
            direction: east;
        }
    }
}

```

```

    arrange_in_box { block_inst A; location: x = 1, y = 0.5; }
    arrange_in_box { group GRP1; location: x = 1, y = 0.5; }
    arrange_in_array {
        contents: block_inst H, block_inst X;
        direction: east;
    }
}

```

#### G. Boundary constraint

To consider the current boundary of a region as rigid, define the boundary as rigid within that scope, and provide a shape. Rigid regions will still be moved to a suitable location, but their shapes will not be changed.

For example,

```

voltage_area VA1 {
    // Only move boundary of VA1, do not change shape
    boundary {
        type: rigid;
        shape: {{0 0} {300 600}};
        // or rectilinear shape as follows
        // shape: {{0 0} {{600 0} {600 1100}
        // {300 1100} {300 500} {0 500} {0 0}};
    }
}

```

#### H. Allowed orientation constraint

With this constraint, the orientation of individual MIB instances can be restricted to a set of allowed orientations. The specified orientations can be R0, R180, MX and MY.

The specified values are expected to be a subset of the allowed orientations of the reference designs of the given block instances. If a reference design has an associated block grid, the allowed orientations follow from that grid. Otherwise, they follow from the design's allowable\_orientations attribute.

If the specified set of orientations is not a subset, it will be ignored silently.

Here's an example restricting the orientation of MIB instances B1, B2, B3 and B4.

```

block Top {
    // Only move boundary of VA1, do not change shape
    allowed_orientation {
        block_inst B1, block_inst B2; // MIB block instances being constrained.
        { R0 R180 }; // Allowed orientations are R0 and R180.
    }
    allowed_orientation {
        block_inst B3, block_inst B4;
        { MY };
    }
}

```

#### -incremental congestion\_driven | target\_utilization\_driven

Specifies that congestion-driven or target utilization-driven incremental shaping is applied. By default, neither shaping strategy is used.

If you specify -incremental congestion\_driven, routing-based congestion information must be available for the incremental shaping to get the initial channel congestion. You must run global routing before using this option. For the congestion driven incremental shaping, all MIBs will not be moved. Only channel congestion between blocks are considered in the incremental shaping.

If you specify -incremental target\_utilization\_driven, the tool incrementally changes the block shapes to achieve the target utilization specified for each block. If no target utilization is specified, the command uses the design utilization as the target for all blocks with respect to the utilization slack specified by set\_shaping\_options -utilization\_slack. For the target area driven incremental shaping, option -channel false is not supported.

This option and option -constraint\_file are mutually exclusive.

Only one of them can be specified.

#### **-pg\_strategy pg\_strategy\_name**

Specifies the PG (power-ground) strategy that the shape\_blocks command should respect so that the shape boundaries are not collinear with PG meshes.

If a mesh pattern, for example my\_pattern, is created using create\_pg\_mesh\_pattern, the PG strategy named smesh can be created with the following command:

```
set_pg_strategy smesh -pattern {{name: my_pattern}{nets: VDD VSS}} -core
```

When you run the shape\_blocks command as follows and specify the smesh PG strategy:

```
shape_blocks -pg_strategy {smesh}
```

the tool creates shapes which are not collinear with the specified PG meshes. You must specify -core with the set\_pg\_strategy command for shaping to create the same PG meshes for the whole design.

#### **-host\_options**

Specifies that the given host option should be used for distributed processing. Note that only creating the appropriate block views (if needed) will use distributed processing in this command.

### **DESCRIPTION**

This command shapes and places physical blocks, power domains or voltage areas, and move bounds with respect to constraints and utilization requirements. The command also applies optimization to minimize feedthrough counts and interface wirelengths. If there is a power domain which has no corresponding voltage area, this command will create its associated voltage area. If the block grid is created for the block before shaping, it will also align the block shape to the specified block grid.

The set\_editability command can be used to allow or prevent shaping on specific blocks. Use the set\_shaping\_options command to specify additional constraints for shaping, such as minimum channel size, channel blockages, and so on.

The tool automatically infers the nesting of move bound inside voltage area based on the contents such that voltage area gets the higher priority. If the elements of voltage area are top/hier1 and the elements of move bound are top/hier1/hier2, the move bound will be nested inside the voltage area based on the hierarchy tree such that all elements of the voltage area are in the same voltage area region. For this reason, it is not supported that the same element is used to define voltage area and move bound, i.e., both use top/hier1. In case that the contents are really the same, use top/hier1/\* to define the elements of move bound.

This command returns 1 on success, 0 otherwise.

### **EXAMPLES**

The following example creates an abutted floorplan.

```
prompt> shape_blocks -channels false
```

The following example creates a floorplan with channels and also respects the constraints specified in the user\_constraints file.

```
prompt> shape_blocks -constraint_file user_constraints
```

Example of making manual adjustments to boundaries after automatic block shaping.

```
set_attribute [get_cells BLK1] -name boundary -value {{0 0} {1000 1000}}
set_attribute [get_voltage_area_shapes -of_objects VA2] -name boundary -value {{0 0} {1000 1000}}
set_attribute [get_bound_shapes -of_objects MB3] -name boundary -value {{0 0} {1000 1000}}
```

### **SEE ALSO**

[create\\_grid\(2\)](#)  
[get\\_bound\\_shapes\(2\)](#)  
[get\\_voltage\\_area\\_shapes\(2\)](#)  
[set\\_editability\(2\)](#)  
[set\\_shaping\\_options\(2\)](#)



